

Topic 5: Simplifying Databases with ORMs

So far in our back-end journey, we've been communicating with our PostgreSQL database by writing SQL queries as strings and sending them using the `node-postgres` library. This works perfectly well, but as applications grow, managing all that SQL can get repetitive and tricky.

What if we could interact with our database using familiar JavaScript objects and methods, without writing a single line of SQL? That's exactly what an ORM allows us to do!

What is an ORM?

ORM stands for **Object-Relational Mapping**. It's a programming technique that acts as a translator or a bridge between the "object-oriented" world of our application (like JavaScript classes and objects) and the "relational" world of a database (like SQL tables and rows).

Think of it like a diplomat. Your Node.js application speaks "JavaScript," and your PostgreSQL database speaks "SQL." The ORM is the expert diplomat in the middle who translates your JavaScript commands into perfect SQL and translates the database's SQL responses back into clean JavaScript objects you can work with.

What Does an ORM Do?

An ORM handles the messy work of database interaction for us. Specifically, it:

- **Maps Tables to Models:** It maps a database table (e.g., a `users` table) to a "Model" in our code (e.g., a `User` class).
- **Maps Rows to Objects:** It converts rows from that table into instances of our `User` class, which are just regular JavaScript objects.
- **Maps Columns to Properties:** It maps the columns of the table (`first_name`, `email`, etc.) to properties on those JavaScript objects (`user.firstName`, `user.email`).
- **Abstracts Queries:** It gives us methods like `.create()`, `.findAll()`, `.update()`, and `.destroy()` to perform CRUD operations, generating the necessary `INSERT`, `SELECT`, `UPDATE`, and `DELETE` SQL behind the scenes.

Benefits vs. Disadvantages

Using an ORM is a trade-off, and it's important to understand both sides.

The Benefits (Why We Use Them)

- **Increased Productivity:** You write less code, faster. Instead of crafting a full INSERT statement, you might just write `User.create({...})`. This speeds up development significantly.
- **Better Readability:** Code like `post.save()` is often more intuitive and easier to maintain than a long, embedded SQL string.
- **Improved Security:** This is a big one! ORMs are designed to prevent **SQL injection attacks**. They automatically sanitize inputs or use parameterized queries, which is a major security advantage over manually building SQL strings with user input.
- **Database Agnosticism:** A well-designed ORM can allow you to switch your underlying database (e.g., from PostgreSQL to MySQL or SQLite) with very few code changes. The ORM handles the small syntax differences between database systems.

The Disadvantages (What to Watch Out For)

- **Performance Overhead:** The abstraction isn't free. The ORM has to generate SQL, and sometimes that generated query isn't as perfectly optimized as one you could write by hand. For most standard operations, this difference is tiny, but for extremely complex, high-performance queries, raw SQL can be faster.
- **"Cost" of Learning:** You have to learn the ORM's own API and way of doing things. It's another layer of abstraction to master. This initial time investment is a form of "cost."
- **Hiding the Complexity:** Because it feels like "magic," it can sometimes be hard to debug a problem if you don't understand the SQL that the ORM is generating. It's still crucial to know what's happening under the hood!

How It Works: An Example with Drizzle ORM

For the Node.js ecosystem, a popular modern choice is **Drizzle ORM**. It's not a traditional ORM like Sequelize; instead, it's a type-safe **query builder**. This means it helps you write SQL queries using TypeScript/JavaScript in a way that's very safe and efficient, but it feels much closer to writing actual SQL.

Let's see how it would simplify the Blog API we designed in the last module.

Imagine we want to get all posts from our `posts` table.

1. The "Old Way" (Module 4 - Raw

node-postgres)

This part stays the same. We are using a raw SQL string to fetch the data.

JavaScript

```
// From our Module 4 approach
app.get('/posts', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM posts');
    res.json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).send('Server Error');
  }
});
```

2. The "New Way" (with Drizzle ORM)

With Drizzle, you first define your database **schema** in a separate file. This is the single source of truth for your table structures.

JavaScript

```
// db/schema.js
import { pgTable, serial, varchar, text } from 'drizzle-orm/pg-core';

// This object defines the 'posts' table, its columns, and their types.
export const posts = pgTable('posts', {
  id: serial('id').primaryKey(),
  title: varchar('title', { length: 255 }).notNull(),
  content: text('content'),
  // created_at from Module 4 could also be added here
});
```

Next, you connect Drizzle to your existing **node-postgres** pool. Drizzle doesn't manage the connection itself; it just uses the driver you provide. Now, your route handler becomes much cleaner and, importantly, **type-safe**.

JavaScript

```
// With our Drizzle query builder
import { drizzle } from 'drizzle-orm/node-postgres';
import { Pool } from 'pg';
```

```
import * as schema from './db/schema';

// Use the same pool from the 'old way'
const pool = new Pool({ connectionString: '...' });
const db = drizzle(pool, { schema });

app.get('/posts', async (req, res) => {
  try {
    // Drizzle's syntax mirrors SQL very closely
    const allPosts = await db.select().from(schema.posts);
    res.json(allPosts);
  } catch (err) {
    console.error(err);
    res.status(500).send('Server Error');
  }
});
```

Notice how `db.select().from(schema.posts)` is very readable and directly translates to `SELECT * FROM posts`. Drizzle gives you the safety and autocompletion benefits of an ORM without hiding the underlying SQL.

✓ Your Task

✓ Your Task (Hard Mode with Drizzle ORM)

Project: Refactoring the Blog API with Drizzle and Adding Relationships

Your mission is to upgrade the Blog API you built in Module 4 to use the **Drizzle ORM**. You'll then leverage Drizzle's powerful relational queries to build connections between users and their posts.

Requirements:

1. Setup & Refactor

- **Install Drizzle:** Install Drizzle and its required drivers (`npm install drizzle-orm pg` and `npm install -D drizzle-kit typescript @types/pg`). Drizzle Kit is used to manage and generate your database schema changes.
 - **Define Schema:** Create a `db/schema.{ts/js}` file. In this file, define your `users` and `posts` tables using Drizzle's schema syntax (`pgTable`, `serial`, `varchar`, etc.) to match the structure from Module 4.
 - **Set Up Connection:** Create a file to initialize your `node-postgres` pool and wrap it with Drizzle, exporting the `db` instance for use in your routes.
 - **Refactor Routes:** Go through all your existing API routes (`GET /posts`, `POST /posts`, etc.) and replace the raw `pool.query()` calls with the equivalent Drizzle query builder methods (`db.select().from(...)`, `db.insert(...).values(...)`, etc.).
-

2. Establish Relationships

- **Define Relations:** In your `db/schema.{ts/js}` file, after defining the tables, use Drizzle's `relations` helper to define the **one-to-many** relationship: A `User` can have many `Posts`, and a `Post` belongs to one `User`.
 - **Add Foreign Key:** This requires adding a `userId` column to your `posts` table that references the `id` in the `users` table.
-

3. Enhance API Functionality

- **Associate Posts with Users:** When a user creates a new post (`POST /posts`), extract their `userId` from the JWT payload (which you set up in Module 4). Use this

`userId` when you call `db.insert(posts).values(...)` to automatically link the new post to the logged-in user.

Include Author Data (Eager Loading): Modify the `GET /posts` and `GET /posts/:id` routes. Use Drizzle's relational query feature to fetch posts and include the author's data in the same query. The result should include the user's email with each post.

- **Secure the Endpoints (Key Challenge):** Update the `PUT /posts/:id` and `DELETE /posts/:id` routes to enforce ownership.
 1. Get the `userId` from the JWT.
 2. First, query for the post the user is trying to modify/delete using `db.query.posts.findFirst(...)`.
 3. Check if the `post.userId` from the database matches the `userId` from the token.
 4. If they **do not match**, send back a **403 Forbidden** error with a message like `"You are not authorized to perform this action."`
 5. If they match, proceed with the update or delete operation.

This exercise will give you hands-on experience with Drizzle's schema-first, type-safe approach to database interactions, which is a very powerful and modern way to build back-end applications. Good luck!